specdec-bench: EAGLE-3 speculative decoding on Llama-3.1-8B-Instruct

Harness for the "Should draft tree shape depend on the workload?" project. Two code paths:

Path	Engine	Batch sizes	Use it for
A	SGLang (pinned tag)	1, 8, 32	Every throughput claim; the matched-budget sweep; SGLang auto and adaptive baselines
B	Official EAGLE repo (pinned commit)	1	Exact reference runs, AR baseline from the same model object, losslessness checks, later TALON / static templates

Target `meta-llama/Llama-3.1-8B-Instruct` (gated: accept the licence on Hugging Face and run `huggingface-cli login`). Drafters: `yuhuili/EAGLE3-LLaMA3.1-Instruct-8B` (official, used by Path B) and `lmsys/sglang-EAGLE3-LLaMA3.1-Instruct-8B` (the pair SGLang's own CI tests, used by Path A).

What was verified, and against what

Every model-facing call and CLI flag in this repo was checked against source, and `tests/` re-checks it:

- **EAGLE repo** `SafeAILab/EAGLE@cb7e0841` (19 Feb 2026): `EaModel.from_pretrained`, `eagenerate`, `naivegenerate` signatures and return values; tree construction in `cnets.py` and `cnets1.py`; token accounting in `utils.py`; the system prompt (copied verbatim, checked by AST).
- **SGLang** `v0.5.22` (CUDA 13) and `v0.5.19` (last CUDA 12 release): every server flag the launcher emits, tree semantics in `eagle_utils.py`, the topk=1 rule and Llama auto-defaults in `speculative_hook.py`, `meta_info` fields in `tokenizer_manager.py`, and `step_time_dict`.
- **transformers**: EAGLE's model code imports `transformers.utils.LossKwargs` (via `modeling_qwen3_kv.py`, loaded even for Llama). It exists in 4.53.3 and is gone in 4.54.0, so the EAGLE environment pins `transformers==4.53.3`. EAGLE's own `requirements.txt` (`>=4.53.1`) would install a version where `import EaModel` fails.

Not verified here: nothing was executed on a GPU, because the workspace that wrote this had no GPU and no package-registry access. The first step on your machine is the smoke test below.

Tree parameters: one vocabulary for both engines

This repo / SGLang	EAGLE repo	Meaning
`steps` (`--speculative-num-steps`)	`depth = steps - 1`	tree depth in draft layers
`topk` (`--speculative-eagle-topk`)	`top_k`	children per parent and parents kept per layer
`budget` (`--speculative-num-draft-tokens`)	`total_token`	verified nodes **including** the root

Candidate pool = `topk + (steps-1)*topk^2`; a config is valid only if `budget - 1 <= pool` (otherwise `torch.topk` fails). With `topk=1`, SGLang forces `budget = steps + 1`. `TreeConfig.validate()` enforces both. The EAGLE repo's eval default (`total_token=60, depth=5, top_k=10`) is `steps=6, topk=10, budget=60` here; TALON's EAGLE-3 baseline (K=10, D=8) is `steps=9`. SGLang's auto choice for Llama is `(5, 4, 8)`.

Setup

```bash
git clone <this repo> specdec-bench && cd specdec-bench
nvidia-smi                          # note the "CUDA Version" (driver) in the top-rig
bash envs/setup_eagle_env.sh        # Path B -> .venv-eagle, clones EAGLE at the pinn
SGLANG_TAG=v0.5.22 bash envs/setup_sglang_env.sh   # Path A; use v0.5.19 on a CUDA
export EAGLE_DATA=$PWD/third_party/EAGLE/eagle/data   # the six question files (80
```

Use separate environments: SGLang v0.5.22 needs torch 2.14.1 and transformers 5.19; the EAGLE repo needs torch 2.6.0 and transformers 4.53.3.

Running on Google Colab

`notebooks/colab_specdec.ipynb` runs everything on a Colab GPU runtime: upload `specdec-bench.zip` to `MyDrive/specdec/`, add `HF_TOKEN` in Colab Secrets, pick an **A100** runtime (40 GB is enough; an L4 handles Path B only), and run the cells top to bottom. The SGLang cell picks `v0.5.22` or `v0.5.19` from the driver's CUDA version.

Checkpoints and saving to disk

Run checkpoints (automatic). `EagleRunner.run` appends to `<out>.jsonl.partial`, flushes and fsyncs after every question, and renames to `<out>.jsonl` only when the file is complete. Re-running the same command continues an interrupted file from the next unfinished

question: a question is kept only if all its turns were written, a line cut off by a crash is dropped, and a checkpoint written with different settings (`_meta`: model, tree, TALON parameters, GPU name, library versions) is discarded so timings are never mixed. `--no-resume` forces a fresh start; `--skip-existing` (EAGLE and SGLang scripts) skips finished files. SGLang files are short (one bench x batch size) and are rewritten whole.

Model checkpoints, code and environments (`scripts/persist.py`), for machines whose local disk is wiped (Colab) — any persistent directory works as `--store`:

```bash
python scripts/persist.py save    --store /content/drive/MyDrive/specdec --what mo
python scripts/persist.py restore --store /content/drive/MyDrive/specdec --what en
python scripts/persist.py status  --store /content/drive/MyDrive/specdec
```

- `models`: each repo's Hugging Face cache entry as one tar (~17 GB total for Llama-3.1-8B and both EAGLE-3 drafts; Llama's duplicate `original/` checkpoint is excluded). Restore puts it back in the HF cache, so every script keeps using the normal repo ids.
- `code`: the repo without venvs, `third_party`, results and caches (previous copy kept as `.prev.zip`).
- `envs`: `.venv-eagle` + `third_party/EAGLE`, `.venv-sglang` + `third_party/sglang`. Restored only into the same path with the same Python version (venvs hard-code both); otherwise the setup scripts reinstall.
- Every item is written as `.partial` and renamed when complete. Free Google Drive (15 GB) is too small for the models.

A laptop (e.g. Apple-silicon Mac) cannot run the experiments (no CUDA; fp16 Llama-3.1-8B needs 16 GB for weights alone), but `python -m unittest discover -s tests` and `scripts/summarize.py` on copied results run fine there.

Day-1 smoke tests

```bash
# Path B: one prompt, EAGLE-3 vs plain decoding, prints tau and speedup
source .venv-eagle/bin/activate
python scripts/quickstart_eagle3.py

# Path A: serve with EAGLE-3 and send one request
source .venv-sglang/bin/activate
bash scripts/quickstart_sglang.sh
```

Expect identical outputs at T=0 (fp16 near-ties can occasionally flip a token) and `tau` around 5-7 for this pair at budget 60 (TALON reports 5.4-7.1 MAT across the six benchmarks).

Running experiments

```bash
# Path B, batch 1: a tree on all six benchmarks, plus AR on the same model load
python scripts/run_eagle.py --data-dir $EAGLE_DATA --bench all --with-ar \
    --steps 6 --topk 10 --budget 60 --out-dir results/eagle/s6_k10_b60 --eagle-rep
python scripts/check_lossless.py results/eagle/s6_k10_b60/mt_bench__s6_k10_b60__T0
                                 results/eagle/s6_k10_b60/mt_bench__ar__T0.jsonl

# Path A: smoke config first (2 servers, 10 questions), then the full sweep (21 se
python scripts/run_sglang_sweep.py --config configs/smoke.yaml --data-dir $EAGLE_D
python scripts/run_sglang_sweep.py --config configs/sweep_eagle3_llama31.yaml --da
    --out-dir results/sglang --skip-existing
python scripts/run_sglang_sweep.py --config configs/sweep_eagle3_llama31.yaml --da

# One CSV with throughput, tau, speedup vs AR (and a bootstrap CI at batch 1)
python scripts/summarize.py results/ --out results/summary.csv
```

`--subset calib|test` (Path B) or `run.subset` (Path A) selects the frozen 30/70 split (hash of bench + question id).

TALON (adaptive draft trees)

`specdec/talon.py` implements TALON (Liu et al., arXiv 2601.07353, Algorithm 2) as a drop-in replacement for EAGLE-3's tree builder `ea_layer.topK_genrate`; verification is unchanged, so decoding stays lossless. Path B only (the paper evaluates batch size 1; SGLang has no hook for it).

- Budget `N` counts the root (Algorithm 2 starts with `T = {x_t}`), so `N` equals EAGLE `total_token` / SGLang `num_draft_tokens`.
- Layer 0: forced top-K children of the root (K = 10). Layers d >= 1: keep every child whose cumulative probability is at least `mu * m_d`, where `m_d` is the best candidate in the layer (mu = 0.03); the budget check (keep the top `N - |T|`) applies at every layer, and every kept node is expanded next. The loop ends exactly when `|T| = N`.
- Children of each parent are pre-filtered to its top `min(N - |T|, vocab)` before gating, which provably returns the same tree as scanning the full vocabulary (argument in the module docstring).

- Context pass, positions, hidden routing, draft-to-target token map and the tree-attention mask update are the same operations as EAGLE's builder (pinned by tests against `cnets.py`).

```bash
python scripts/quickstart_talon.py                          # one prompt: plain vs E
python scripts/run_eagle.py --data-dir $EAGLE_DATA --bench all --talon --budget 60
    --root-k 10 --out-dir results/eagle/talon_n60_mu0.03_k10
EAGLE_DATA=$EAGLE_DATA bash scripts/run_talon_suite.sh    # paper setting + N and
```

The paper's EAGLE-3 baseline is K=10, D=8, N=60. Its Algorithm 1 runs D+1 = 9 draft layers, which is EAGLE-repo `depth=8`, i.e. `--steps 9 --topk 10 --budget 60` here (labelled `talon_eagle3_baseline` in the plan grid).

Both static and TALON runs now record `draft_forwards`, `delta` (draft passes per verify pass, TALON's draft efficiency; it includes the tree built right after prefill), `mean_tree_depth` and `mean_tree_nodes` per turn; `summarize.py` adds `delta` and `mean_tree_depth` columns.

Verification done without a GPU: an independent full-vocabulary reference of Algorithm 2 must produce the identical tree (same nodes, same order) on 380+ random configurations; a fake EAGLE-3 draft layer reconstructs every node's path only from the attention mask and KV it is given and checks hidden routing and positions; outputs are compared with a literal port of EAGLE's retrieve-index code; 14 deliberately injected bugs (wrong anchor, missing root top-K, broken mask inheritance, polluted draft cache, ...) are all caught. When torch is installed, the same tests also run on `TorchOps`, so run `python -m unittest tests.test_talon -v` in `.venv-eagle` before the first real run.

Measurement definitions

- `tau` = generated tokens / verify passes, bonus token included (EAGLE `new_token/(idx+1)`, SGLang `spec_accept_length`).
- Throughput = output tokens (cut at the first stop token) / wall seconds. Path A reports two values: over the whole run, and over the window where all `b` slots were busy (`throughput_full_occupancy`, used by `summarize.py`). It also records SGLang's own per-step decode time for that batch size (`server_step_time`).
- Batch size `b` = at most `b` conversations in flight; MT-Bench turns run in order with the previous answer fed back, as in EAGLE's script.
- Radix (prefix) cache is disabled so repeated prompts at `b=32` are not served from cache.

Two traps in the official EAGLE code that this harness avoids

1. The KV cache is allocated once, on the first `eagenerate` / `naivegenerate` call, using that call's `max_length`, and then reused. The runner always passes the same `max_length` (4096), large enough for CNN/DM prompts plus 1,024 new tokens.
2. The official eval script never passes `max_new_tokens`, so it silently stops at 512. The runner passes it explicitly to both speculative and AR generation.

Tests

```bash
python -m unittest discover -s tests -v      # no GPU or torch needed
EAGLE_REPO=third_party/EAGLE SGLANG_REPO=third_party/sglang python -m unittest dis
```

Logic tests use fakes with the real call signatures (a mock HTTP server stands in for SGLang). Source-pin tests read the EAGLE and SGLang checkouts and fail if a flag, signature, tree rule or response field the harness depends on changes; run them after any version bump.